INNODHARA

11 August 2026

Outstanding Dependencies

Five items that need a decision or a resource before wider rollout

The product is functionally complete and demonstrable end to end. What follows is the short list of things it still depends on. **Hosting is solved** — the office VM's 12 GB GPU is more than enough. **One item needs a decision from you** (item 2, which is affecting the product today); the remaining three are engineering work we can schedule.

#	Dependency	State today	Needs
1	GPU host for deployment	Runs on a developer laptop (8 GB GPU)	**Cleared** — the office GPU (12 GB) is enough
2	Sentence embeddings for the notes feature	**Broken.** Google credits exhausted	**Your decision** — pay, or switch to a local model
3	Long-form proposal generation	One page, from a fixed template	Engineering — a new pipeline
4	SQLite → PostgreSQL	SQLite, 1.8 MB, single file	Engineering — plus a migration tool
5	Vectors in a real vector database	JSON column, compared in Python	Engineering — follows 2 and 4

1 A GPU host to deploy the backend on

Only one part of the system needs a GPU: **speech-to-text**. Meeting recordings are transcribed on our own machine with OpenAI Whisper, which is deliberate — the audio never leaves our server and there is no per-minute API cost. The AI reasoning runs on a hosted API and needs no GPU at all.

VERIFIED TODAY

We transcribe meetings with the Whisper **medium** model and field dictation with **tiny**. Development currently runs on a laptop with an **RTX 4060, 8 GB of video memory** — verified: CUDA is active and Whisper is installed. Whisper's own published requirement is **about 5 GB for medium**, ~2 GB for small and ~10 GB for large.

CLEARED FOR DEPLOYMENT

A GPU with 12 GB of video memory is already available on the office VM — can we deploy there? Yes. 12 GB comfortably exceeds the ~5 GB the *medium* model needs, so the backend can move onto that VM as it stands, at no cost and with no loss of transcription quality.

It also buys us an upgrade. At 12 GB the *large* model (~10 GB) fits, which is the one meaningful accuracy gain available to us on accented speech and crosstalk — the two conditions our recordings actually have. We would keep *medium* for the initial cut-over and evaluate *large* on real recordings before switching.

2 Sentence embeddings for the notes feature

Notes about a client are searched by *meaning* rather than by keyword — "what did they say about the handover pack?" finds the right note even when those words were never used. That requires an embedding model, and we currently call Google's **gemini-embedding-001**.

IMPACT RIGHT NOW

This feature is not working today. We tested the live call while writing this: Google returns **429 — "Your prepayment credits are depleted"**. The consequence is worse than slow search. When the embedding call fails, a new note is saved and readable in the app but is **never added to the searchable memory**. The database shows it plainly: **28 client notes stored, 0 indexed**. So semantic search returns nothing at all, and has been quietly doing so since the credits ran out.

DECISION NEEDED

Two ways forward. **(a)** Top up the Google credits — immediate, but a recurring cost that grows with usage, and a client's notes leave our infrastructure. **(b)** Run the embedding model locally, as we already do for speech: a small open model such as **sentence-transformers / all-MiniLM-L6-v2** (about 90 MB, runs on CPU, no per-call cost, nothing leaves the server).

We recommend (b), and it is contained work — the code is written so only one module changes, and the stored format needs no migration because nothing is indexed yet. It also removes the only remaining dependency on Google for day-to-day operation. This pairs naturally with item 1: the same host serves both models.

3 Long-form proposals, written rather than filled in

VERIFIED TODAY

Today a proposal is produced by filling one saved template: the organisation's document with placeholders for the client, the contact, the rate table and the total. The proposal on the system measures **1,171 characters and renders to a single page**. It is correct and fast, but it is a form, not a document — and it can only ever say what the template already says.

The goal is a **20–30 page proposal, composed by the AI from the engagement itself**, with no template: an executive summary, our understanding of the requirement, scope per phase, method, deliverables, the schedule, the team, assumptions, risks, and commercial terms. This is the largest of the five items and genuinely new capability rather than migration work.

CONSTRAINT TO PRESERVE

One rule must survive the change, and it is the reason clients can trust the output: **the AI never authors a number**. Every price and total is computed from the rate card, and a rewrite that slips a different figure into the prose is rejected automatically. A long-form generator has to keep that fence while writing far more text — so the design is section-by-section generation against facts already on file, with the commercial pages assembled, not written. We can scope this properly once items 1 and 2 are settled.

4 SQLite to PostgreSQL

The database is a single SQLite file, **1.8 MB** today. That is the right choice for one team and the wrong one for several organisations at once, because SQLite serialises writers. The code already speaks both dialects, so the move is configuration plus three pieces of real work:

Work	Why it is needed
Add the PostgreSQL driver	Already listed in our dependencies, deliberately switched off
Adopt a schema-migration tool	There is **none today** — the schema is created by the application on start-up, which is safe now but cannot be used once real client data exists
Finish the database-level tenant isolation	A script exists and covers **25 of the 54** client-scoped tables; the remaining **29** need the same policy, and the application must declare which organisation each connection is acting for — that hook is not written yet

> **VERIFIED TODAY**
>
> To be clear about what is and is not at risk: client data is **already isolated** today — every query is filtered by organisation and every row carries its organisation's identifier. The database-level policy is a *second* independent net for the case where a future code change forgets a filter. It is a hardening step, not an open hole.

5 Vectors in a vector database

Embeddings are stored as a plain list of numbers in a normal column, and a search loads **every** note belonging to that organisation and compares them one by one in Python. That is exact and perfectly fast at our scale, and it stops being either at somewhere in the low tens of thousands of notes. The fix is to keep the vectors where a database can index them — the PostgreSQL **pgvector** extension, which arrives free with item 4, or a dedicated service if we outgrow that.

> **VERIFIED TODAY**
>
> This is the least urgent of the five and should be done **last**: it depends on which embedding model we choose in item 2 (that decides the vector size) and on the PostgreSQL move in item 4. Doing it before those two would mean doing it twice. The code was written with this in mind — the same single module owns all of it.

What we need from you

Ask	Why we need it	Cost
Approve running the embedding model locally rather than buying more Google credits	The notes search is dead until one of the two is chosen. This is the only item currently degrading the product — item 2	None, ~2 days of work
Confirm the office VM's GPU is passed through to the guest, and that it has ~6 GB of free disk	Deployment is otherwise cleared. If the GPU is not visible inside the VM, transcription silently runs on the CPU — item 1	None
A slot to scope long-form proposals	The largest item, and worth specifying properly rather than guessing — item 3	One meeting

Items 4 and 5 need no decision — they are scheduled engineering work, best done in that order, and neither blocks a client demonstration today.